# Distributed Webhook Relay & Real-Time Debugger

https://gemini.google.com/app/7d7e0ce45879aac5

*User prompt: Explain me the above as if you are explaining to a kid*

Response: Imagine you are throwing a huge birthday party, and every time someone buys a ticket online, you have to run over and hand an invitation letter to their house.

If you tried to do this all by yourself, you would run into some big problems. This project is like building a **Super-Smart Post Office** with a big glass viewing window to do the hard work for you.

### 1. You don't have to pause your day (No Waiting)

- **The Problem:** If you had to drop what you were doing, run across town to deliver each letter, and wait at the front door until your friend answered, you would be exhausted, and everyone waiting in line behind you would get annoyed.
- **The Fix:** Now, you just drop the letter into a mailbox slot (`ingest-api`). The mailbox says *"Got it!"* in one second, and you can immediately go back to playing with your toys.

### 2. Letters never get lost in the dark (The Locked Safe)

- **The Problem:** What if you held the letter in your hand, tripped, and dropped it into a puddle? Or what if you fell asleep and forgot you were supposed to send it? The letter is gone forever.
- **The Fix:** The post office locks every single letter inside a giant, fireproof diary (`PostgreSQL`) before doing anything else. Even if the power goes out or the building shakes, the letter is safely written down.

### 3. What if your friend isn't home? (Smart Retries)

- **The Problem:** A delivery driver walks up to your friend's house, rings the doorbell, but nobody answers because they are at soccer practice. A silly driver would throw the letter in the trash and give up.
- **The Fix:** The smart delivery driver (`dispatch-worker`) says: *"No problem. I will come back in 5 minutes. If they are still not home, I will try in an hour."* It keeps trying politely until your friend finally opens the door.

### 4. Delivery drivers never bump into each other (No Clashing)

- **The Problem:** If five delivery drivers all rush toward the mail pile at the same time, they might grab the same letter and both run to the same house, giving your friend two identical invitations.
- **The Fix:** The post office hands each driver a different letter one at a time (`SKIP LOCKED`). Nobody fights, nobody bumps into each other, and nobody delivers the same invite twice.

### 5. A giant TV screen on your wall (The Debugger Screen)

- **The Problem:** If your friend calls you crying and says *"I never got my invitation!"*, you have no idea what went wrong. Did the driver get lost? Did the dog eat it?
- **The Fix:** You have a glowing TV dashboard in your bedroom (`debugger-ui`). You can see every letter fly across the screen live:
  - 🟢 **Green light:** "Delivered to Timmy's house in 40 milliseconds!"
  - 🔴 **Red light:** "Sarah's mailbox was full!"
  - And right next to the red light, there is a big **"Try Again"** button so you can send it again with one tap.

---

*User prompt: Thank you, now give me a real life scenario where this can be used, a real world problem*

Response: Here is a concrete, high-stakes real-world scenario where this exact system is required: **A B2B Payment Gateway (like Stripe or Razorpay) during a massive Flash Sale.**

### The Scenario: "The Black Friday / Festival Sale Nightmare"

You run **PayFast**, a payment platform used by 5,000 different e-commerce merchants (clothing brands, electronics stores, and ticket sellers).

Whenever a customer buys something on a merchant's store, PayFast charges the customer's credit card, and PayFast must send an **HTTP Webhook (`payment.succeeded`)** to that merchant's server so they can:

1. Mark the order as paid.
2. Decrement their inventory.
3. Send the order to the warehouse for shipping.

## What Happens Without This System (The Real Problem)

It's 12:00 AM on the biggest shopping day of the year. Traffic spikes to **15,000 transactions per minute**.

#### 1. The Cascading Timeout Disaster

A popular clothing merchant running on cheap hosting experiences an outage because of the traffic surge. Their server starts taking **30 seconds to respond**, or throws `504 Gateway Timeout`.

- If PayFast tries to send the webhook directly during the checkout request, PayFast's own checkout servers freeze waiting for the merchant's broken server to answer.
- PayFast runs out of HTTP connection sockets, and checkout completely freezes for **all** merchants, not just the broken one.

#### 2. The "Lost Money, No Product" Disaster

PayFast charges a customer's card for a $1,200 laptop. A fraction of a second later, PayFast's server hits a memory limit and restarts before the webhook is sent:

- The customer's bank account is charged $1,200.
- The merchant **never** gets notified that the payment succeeded.
- The merchant never ships the laptop.
- Two days later, an angry customer files a credit card dispute, and the merchant loses money and trust.

#### 3. The Support Nightmare

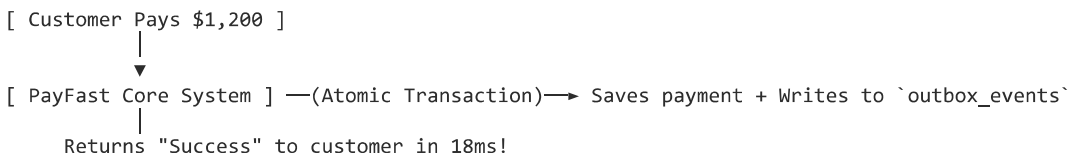
The next morning, hundreds of merchants flood PayFast's customer support:

> *"My store had 800 sales last night, but my inventory system only shows 300! Did you drop our webhooks, or did our server miss them? Where are the missing 500 orders?!"*

PayFast engineers now have to spend hours searching through gigabytes of raw, cryptic server logs trying to figure out which payloads failed and why.

## How the Webhook Relay Solves This Real-World Problem

With the **Distributed Webhook Relay & Real-Time Event Debugger**, the entire scenario changes:




#### 1. Guaranteed Delivery via Transactional Outbox

The event is saved in PostgreSQL in the **same millisecond** the payment record is saved. It is physically impossible for the payment to succeed without the webhook being safely queued. Even if PayFast servers get restarted or crash, the event is sitting safely in PostgreSQL.

#### 2. The Clothing Store Server Stays Down? Exponential Backoff Kicks In

When the clothing store's server returns `500 Server Error`, the `dispatch-worker`:

- Does **not** drop the webhook.
- Does **not** block other stores' deliveries.
- Automatically schedules a retry: 1st retry in 1 minute, 2nd retry in 5 minutes, 3rd retry in 30 minutes.
- When the clothing merchant finally fixes their server at 2:00 AM, the worker automatically delivers all pending orders smoothly without flooding them.

#### 3. Zero Race Conditions across Worker Pods

During peak hours, PayFast scales up from 2 worker pods to 20 worker pods. Because they query using `SELECT ... FOR UPDATE SKIP LOCKED`, all 20 worker pods grab different chunks of pending webhooks simultaneously without locking each other out or double-charging/double-shipping orders.

#### 4. The Live Debugger UI Solves the Support Crisis

Instead of calling support, merchants log into their PayFast Developer Portal (powered by **Leptos + Tailwind CSS**):

- They see a live feed showing every delivery attempt.
- The rows for the downed server clearly show 🔴 `504 Gateway Timeout` with exact request and response headers and timestamp logs.
- Once their developer fixes their endpoint, they don't have to wait for the automatic retry: they click a single button—**"Replay Failed Deliveries"**—and watch the rows immediately flip to 🟢 `200 OK` in real time.

### Commercial Precedents

This is not just a theoretical exercise—entire multi-million dollar companies and services are built specifically around this architecture:

- **Stripe's Webhook Infrastructure:** Powers millions of daily financial events with cryptographic signing and automated retry policies.
- **Svix & Hookdeck:** Dedicated "Webhooks-as-a-Service" platforms that companies pay thousands of dollars per month to handle this exact queue-and-relay problem.
- **Shopify & GitHub:** Use identical transactional outbox queues to push order updates and repository events to third-party app developers.

---

*User prompt: i have considered to build this project : Distributed Webhook Relay & Real-Time Event Debugger*

### Response: Implementation Roadmap

Building the **Distributed Webhook Relay & Real-Time Event Debugger** breaks down into six progressive phases:

```
Phase 1: Workspace Scaffolding & Shared Crates
   ↓
Phase 2: Database Schema & Ingestion API (Axum)
   ↓
Phase 3: Tokio Worker Dispatch Engine (SKIP LOCKED + Retries)
   ↓
Phase 4: Stream Bridge Service (PostgreSQL LISTEN/NOTIFY → SSE)
   ↓
Phase 5: Reactive Web Debugger (Leptos + Tailwind CSS)
   ↓
Phase 6: End-to-End Load Testing & Failure Simulation
```

### Step 1: Workspace Configuration (`Cargo.toml`)

Create a root directory and define the workspace manifest. Centralizing dependencies in `[workspace.dependencies]` prevents version mismatches across microservices and shared libraries.

```
mkdir webhook-relay && cd webhook-relay
git init
```

Create `Cargo.toml` at the project root:

```
[workspace]
resolver = "2"
members = [
    "crates/core-types",
    "crates/crypto-signer",
    "crates/db-store",
    "services/ingest-api",
    "services/dispatch-worker",
    "services/stream-bridge",
    "frontend/debugger-ui"
]

[workspace.dependencies]
# Async Runtime & Utilities
tokio = { version = "1.38", features = ["full"] }
tokio-util = { version = "0.7", features = ["codec"] }
futures = "0.3"

# Database & Migrations
sqlx = { version = "0.8", default-features = false, features = [
    "runtime-tokio-rustls",
    "postgres",
    "uuid",
    "chrono",
    "json",
    "macros"
] }
```

```
# Serialization & Web
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
axum = { version = "0.7", features = ["macros"] }
tower = "0.4"
tower-http = { version = "0.5", features = ["cors", "trace"] }
reqwest = { version = "0.12", default-features = false, features = ["json", "rustls-tls"] }

# Identifiers, Time & Crypto
uuid = { version = "1.8", features = ["v7", "serde"] }
chrono = { version = "0.4", features = ["serde"] }
hmac = "0.12"
sha2 = "0.10"
hex = "0.4"

# Observability
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter", "fmt"] }
thiserror = "1.0"
dotenvy = "0.15"

# Frontend (Leptos CSR)
leptos = { version = "0.6", features = ["csr"] }
leptos_meta = { version = "0.6", features = ["csr"] }
leptos_router = { version = "0.6", features = ["csr"] }
gloo-net = "0.5"
web-sys = "0.3"
```

### Step 2: Initialize Directory Structure

Run these commands in the project root to generate the crates and services:

```
# Shared crates
cargo new --lib crates/core-types
cargo new --lib crates/crypto-signer
cargo new --lib crates/db-store

# Backend microservices
cargo new --bin services/ingest-api
cargo new --bin services/dispatch-worker
cargo new --bin services/stream-bridge

# Frontend client (Leptos)
cargo new --lib frontend/debugger-ui
```

### Step 3: Shared Core Types (`crates/core-types/src/lib.rs`)

Define the domain primitives shared across all ingestion, dispatching, and streaming services.

```
use chrono::{DateTime, Utc};
use serde::{Deserialize, Serialize};
use uuid::Uuid;

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq, sqlx::Type)]
#[sqlx(type_name = "text", rename_all = "SCREAMING_SNAKE_CASE")]
pub enum DeliveryStatus {
    Pending,
    Delivered,
    Failed,
    DeadLetter,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct IngestWebhookRequest {
    pub destination_url: String,
    pub payload: serde_json::Value,
    pub secret_key: String,
}

#[derive(Debug, Clone, Serialize, Deserialize, sqlx::FromRow)]
pub struct OutboxEvent {
    pub id: Uuid,
    pub destination_url: String,
```

```
    pub payload: serde_json::Value,
    pub secret_key: String,
    pub status: String,
    pub attempt_count: i32,
    pub max_attempts: i32,
    pub next_retry_at: DateTime<Utc>,
    pub created_at: DateTime<Utc>,
}

#[derive(Debug, Clone, Serialize, Deserialize, sqlx::FromRow)]
pub struct DeliveryAttempt {
    pub id: Uuid,
    pub event_id: Uuid,
    pub http_status: Option<i32>,
    pub response_body: Option<String>,
    pub latency_ms: i32,
    pub error_message: Option<String>,
    pub attempted_at: DateTime<Utc>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct LiveEventNotification {
    pub attempt_id: Uuid,
    pub event_id: Uuid,
    pub http_status: Option<i32>,
    pub latency_ms: i32,
    pub destination_url: String,
    pub attempted_at: DateTime<Utc>,
}
```

### Step 4: Cryptographic Signer (`crates/crypto-signer/src/lib.rs`)

External endpoints require verification that webhook payloads originated from your system and were not tampered with in transit.

```
use hmac::{Hmac, Mac};
use sha2::Sha256;

type HmacSha256 = Hmac<Sha256>;

pub fn generate_signature(payload: &str, secret: &str, timestamp: i64) -> Result<String, String> {
    let mut mac = HmacSha256::new_from_slice(secret.as_bytes())
        .map_err(|e| format!("Invalid HMAC key: {e}"))?;

    // Stripe-style signature envelope: timestamp.payload
    mac.update(format!("{timestamp}.{payload}").as_bytes());
    let result = mac.finalize();
    Ok(hex::encode(result.into_bytes()))
}
```

### Step 5: Database Setup & SQLx Migration

Install the SQLx CLI and run the schema setup:

```
cargo install sqlx-cli --no-default-features --features native-tls,postgres
mkdir -p crates/db-store/migrations
```

Create `crates/db-store/migrations/20260101000000_init_schema.sql`:

```
CREATE TABLE outbox_events (
    id UUID PRIMARY KEY,
    destination_url TEXT NOT NULL,
    payload JSONB NOT NULL,
    secret_key TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'PENDING',
    attempt_count INT NOT NULL DEFAULT 0,
    max_attempts INT NOT NULL DEFAULT 5,
    next_retry_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_outbox_queue ON outbox_events (next_retry_at ASC)
WHERE status = 'PENDING';

CREATE TABLE delivery_attempts (
```

```
    id UUID PRIMARY KEY,
    event_id UUID NOT NULL REFERENCES outbox_events(id) ON DELETE CASCADE,
    http_status INT,
    response_body TEXT,
    latency_ms INT NOT NULL,
    error_message TEXT,
    attempted_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Trigger to emit events on delivery attempts for real-time streaming
CREATE OR REPLACE FUNCTION notify_delivery_attempt()
RETURNS TRIGGER AS $$
DECLARE
    target_url TEXT;
BEGIN
    SELECT destination_url INTO target_url FROM outbox_events WHERE id = NEW.event_id;

    PERFORM pg_notify(
        'webhook_stream',
        json_build_object(
            'attempt_id', NEW.id,
            'event_id', NEW.event_id,
            'http_status', NEW.http_status,
            'latency_ms', NEW.latency_ms,
            'destination_url', target_url,
            'attempted_at', NEW.attempted_at
        )::text
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_delivery_attempt
AFTER INSERT ON delivery_attempts
FOR EACH ROW EXECUTE FUNCTION notify_delivery_attempt();
```

### Recommended Development Sequence

| Phase | Milestone | Primary Crate / Tool |
|---|---|---|
| **1. Persistence** | Run local Postgres (`docker-compose.yml`), execute `sqlx migrate run`. | `crates/db-store`, Docker |
| **2. Ingestion** | Build `ingest-api` using Axum to accept POST requests and write to `outbox_events`. | `services/ingest-api` |
| **3. Dispatch Worker** | Implement the `SELECT ... FOR UPDATE SKIP LOCKED` poll loop with `FuturesUnordered`. | `services/dispatch-worker` |
| **4. Event Stream** | Implement `PgListener` in `stream-bridge` and expose an SSE endpoint (`/events/stream`). | `services/stream-bridge` |
| **5. Frontend UI** | Scaffold the Leptos dashboard with Tailwind CSS to render live SSE notifications. | `frontend/debugger-ui` |

Would you like to write the **Axum Ingestion API** and `sqlx` repository queries next, or focus on the **Tokio Dispatch Worker** polling loop?

---

*User prompt: # Backend microservices cargo new --bin services/ingest-api cargo new --bin services/dispatch-worker cargo new --bin services/stream-bridge which one is the main bin and hwo to run*

Response: In a microservices architecture, **none of them is the single "main" binary**—all three are independent executables designed to run simultaneously as separate processes.

Each service has a distinct operational lifecycle:

- **`services/ingest-api` (Ingress Gateway):** Runs an HTTP server (Axum) listening on a public port (e.g., `:3000`) to ingest incoming webhooks and write them to PostgreSQL.
- **`services/dispatch-worker` (Background Daemon):** Runs a continuous asynchronous polling loop with Tokio. It has no open HTTP server ports—it exclusively queries PostgreSQL for pending outbox jobs and sends outbound HTTP requests to destination APIs.
- **`services/stream-bridge` (Telemetry Gateway):** Runs an HTTP server on a separate port (e.g., `:3001`) listening for database notifications via `LISTEN/NOTIFY` and streaming them to the Leptos frontend over Server-Sent Events (SSE).

### How to Run Them

Because these packages live inside a Cargo workspace, running `cargo run` from the root directory will prompt an error asking which binary to execute. You must target each package using the `-p` (or `--package`) flag.

**1. Ensure Prerequisites Are Running**

Before starting the Rust binaries, ensure PostgreSQL is running and migrations have been executed:

```
# Start your Postgres instance (e.g., via Docker)
docker run --name pg-relay -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres:16

# Apply migrations
sqlx migrate run --source crates/db-store/migrations
```

**2. Run Each Service in Separate Terminal Tabs**

During local development, open three terminal windows in your project root:

**Terminal 1 (Ingest API):**

```
cargo run -p ingest-api
```

**Terminal 2 (Dispatch Worker):**

```
cargo run -p dispatch-worker
```

**Terminal 3 (Stream Bridge):**

```
cargo run -p stream-bridge
```

## Running All Services Concurrently (Recommended Dev Workflows)

Rather than manually launching three separate terminal tabs, you can coordinate them using any of the standard process managers:

**Option A: Using a `Procfile` with `overmind` or `foreman`**

Create a file named `Procfile` in the workspace root:

```
ingest:   cargo run -p ingest-api
worker:   cargo run -p dispatch-worker
stream:   cargo run -p stream-bridge
```

Run all three simultaneously with a unified log stream:

```
overmind start
# or: npx foreman start
```

**Option B: Using a `justfile` or Shell Script**

If you use `just` (`cargo install just`), create a `justfile`:

```
# Run all services concurrently in development
dev:
    cargo build
    trap 'kill 0' EXIT; \
    cargo run -p ingest-api & \
    cargo run -p dispatch-worker & \
    cargo run -p stream-bridge & \
    wait
```

Run it with:

```
just dev
```

**Option C: Production Docker Compose**

In production, compile each binary into a slim scratch/Alpine container and coordinate them inside `docker-compose.yml`:

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: relay
      POSTGRES_PASSWORD: secretpassword
    ports:
      - "5432:5432"
```

```
  ingest-api:
    build:
      context: .
      dockerfile: services/ingest-api/Dockerfile
    ports:
      - "3000:3000"
    depends_on:
      - postgres

  dispatch-worker:
    build:
      context: .
      dockerfile: services/dispatch-worker/Dockerfile
    depends_on:
      - postgres

  stream-bridge:
    build:
      context: .
      dockerfile: services/stream-bridge/Dockerfile
    ports:
      - "3001:3001"
    depends_on:
      - postgres
```

---

*User prompt: cargo new --lib frontend/debugger-ui, if this is a lib crate where do i call it or run it*

Response: A browser WebAssembly (WASM) application does not run as a native desktop or server binary; instead, it compiles into a WebAssembly library (`.wasm` bundle) that is embedded in an HTML file and executed inside the browser's JavaScript engine.

Because the browser runtime—not your terminal—is what executes the code, the crate is compiled as a `cdylib` (C-compatible dynamic library) rather than a standalone CLI executable.

### 1. Configure the Crate Type (`frontend/debugger-ui/Cargo.toml`)

Add `crate-type = ["cdylib", "rlib"]` so compiler tools know to produce a `.wasm` binary that JavaScript can instantiate:

```
[package]
name = "debugger-ui"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib", "rlib"]

[dependencies]
leptos = { workspace = true }
leptos_meta = { workspace = true }
leptos_router = { workspace = true }
wasm-bindgen = "0.2"
console_error_panic_hook = "0.1"
```

### 2. Define the Entry Point (`frontend/debugger-ui/src/lib.rs`)

Use the `#[wasm_bindgen(start)]` macro. This function executes automatically as soon as the browser downloads and initializes the `.wasm` file:

```
use leptos::*;
use wasm_bindgen::prelude::*;

#[component]
pub fn App() -> impl IntoView {
    view! {
        <main class="min-h-screen bg-slate-900 text-slate-100 p-8">
            <h1 class="text-3xl font-bold tracking-tight text-white">
                "Webhook Relay & Event Debugger"
            </h1>
            <p class="mt-2 text-slate-400">
                "Live telemetry connected to stream-bridge via SSE."
            </p>
        </main>
    }
}
```

```
// The browser entry point
#[wasm_bindgen(start)]
pub fn run_app() {
    // Prints Rust panics directly to browser developer console (F12)
    console_error_panic_hook::set_once();

    // Mounts the reactive Leptos tree onto the <body> of index.html
    mount_to_body(|| view! { <App/> });
}
```

### 3. Create the HTML Shell (`frontend/debugger-ui/index.html`)

Create an `index.html` file inside `frontend/debugger-ui/`. The build tool injects the compiled WebAssembly script into this file:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Webhook Debugger UI</title>
    <!-- Trunk watches and compiles your Tailwind CSS input -->
    <link data-trunk rel="tailwind-css" href="style/input.css" />
  </head>
  <body class="bg-slate-900"></body>
</html>
```

### 4. How to Run It: Using `trunk`

Rust frontends use **Trunk**, a dedicated WASM web server and build tool that handles compiling Rust to WASM, bundling CSS, and live-reloading the browser.

#### Install Trunk & the WebAssembly Target

```
cargo install trunk
rustup target add wasm32-unknown-unknown
```

#### Run the Development Server

Navigate into the frontend crate and start Trunk:

```
cd frontend/debugger-ui
trunk serve --open
```

- Trunk compiles `frontend/debugger-ui` using `rustc --target wasm32-unknown-unknown`.
- It generates JavaScript glue code, compiles your Tailwind CSS file, and bundles them into a `/dist` folder.
- It hosts a local web server (defaults to `[http://127.0.0.1:8080](http://127.0.0.1:8080)`) and auto-reloads whenever you edit Rust files.

### Alternative: When is it called by another Rust crate?

If you prefer **Server-Side Rendering (SSR)** instead of a pure client-side SPA:

1. `frontend/debugger-ui` remains a library containing your components (`pub fn App`).
2. Your `services/stream-bridge` (or a dedicated web server binary) adds `debugger-ui = { path = "../../frontend/debugger-ui" }` to its dependencies.
3. The Axum server imports and renders `debugger_ui::App` to an HTML string on the server using `leptos_axum`, serving it directly over port `3001`.

---

*User prompt: attributes avaliable to set in cargo.toml file in workspace*

Response: In a Cargo workspace, settings in the root `Cargo.toml` fall into three main categories: direct `[workspace]` table attributes, inheritable workspace tables, and workspace-wide root tables.

### 1. Direct `[workspace]` Attributes

These keys configure member discovery, execution defaults, and dependency resolution for the workspace itself.

| Attribute | Type | Description |
|---|---|---|
| `members` | `Array<String>` | File paths or glob patterns specifying directories containing member crates (e.g., `["crates/*", "services/*"]`). |
| `exclude` | `Array<String>` | Paths or glob patterns to exclude from being treated as workspace members, even if matched by `members`. |
| `default-members` | `Array<String>` | Explicit subset of member crates to build, run, or test when commands like `cargo build` or `cargo test` are run from the workspace root without `--workspace` or `-p`. |
| `resolver` | `String` | Dependency resolver algorithm version: `"1"` (Rust 2015/2018 default), `"2"` (Rust 2021 default), or `"3"` (introduced in Rust 2024). |

### 2. Inheritable Workspace Tables

These sections allow member crates to share identical versions, metadata, and configurations instead of duplicating them across multiple `Cargo.toml` files.

**`[workspace.package]`**

Defines package metadata that members can inherit by setting `<key>.workspace = true` in their local `[package]` sections.

- `version`
- `edition`
- `rust-version` (MSRV)
- `authors`
- `description`
- `homepage`
- `repository`
- `documentation`
- `readme` (relative to workspace root)
- `license` or `license-file`
- `keywords`
- `categories`
- `publish`
- `include` / `exclude`

**`[workspace.dependencies]`**

Defines canonical dependency versions and default feature flags for all members.

- Members inherit a dependency by writing: `crate_name = { workspace = true }`.
- Members can append crate-specific features: `crate_name = { workspace = true, features = ["extra"] }`.
- Dependencies in this table cannot be marked as `optional = true` (member crates declare `optional = true` locally if needed).

**`[workspace.lints]`**

Defines compiler and Clippy lint groups across all members.

- Sub-tables include `[workspace.lints.rust]`, `[workspace.lints.clippy]`, and `[workspace.lints.rustdoc]`.
- Members activate these rules by adding `[lints] workspace = true` in their local manifests.

**`[workspace.metadata]`**

Arbitrary configuration table ignored by Cargo itself, but used by external ecosystem tools to store workspace-wide policies (e.g., `[workspace.metadata.cargo-deny]`, `[workspace.metadata.release]`).

### 3. Root-Level Workspace Tables

These top-level tables **must** be defined in the workspace root manifest; if declared inside member crates, Cargo will ignore them.

- **`[profile.<name>]` (`dev`, `release`, `test`, `bench`, custom profiles):** Defines optimization levels (`opt-level`), debug info levels (`debug`), link-time optimization (`lto`), codegen units, and panic strategies for all compilation units across the workspace.
- **`[patch.<registry>]` (e.g., `[patch.crates-io]`):** Overrides dependencies globally with local path forks or specific git revisions across all member crates.
- **`[replace]`:** The deprecated precursor to `[patch]`.

### Comprehensive Example

```
[workspace]
resolver = "2"
members = [
    "crates/*",
    "services/*",
    "frontend/debugger-ui"
]
exclude = [
    "target",
    "benches/fixtures"
]
default-members = [
    "services/ingest-api",
    "services/dispatch-worker"
]

# 1. Inherited crate metadata
[workspace.package]
version = "0.1.0"
edition = "2021"
rust-version = "1.80.0"
authors = ["Your Team <team@example.com>"]
license = "MIT OR Apache-2.0"
repository = "https://github.com/org/repo"

# 2. Inherited dependencies
[workspace.dependencies]
tokio = { version = "1.38", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
sqlx = { version = "0.8", default-features = false, features = ["runtime-tokio-rustls", "postgres", "uuid"] }
uuid = { version = "1.8", features = ["v7", "serde"] }
thiserror = "1.0"

# Local workspace crate references
core-types = { path = "crates/core-types" }
db-store = { path = "crates/db-store" }

# 3. Inherited lints
[workspace.lints.rust]
unsafe_code = "forbid"
missing_debug_implementations = "warn"

[workspace.lints.clippy]
unwrap_used = "deny"
dbg_macro = "warn"

# 4. Global compilation profiles
[profile.dev]
opt-level = 0
debug = true

[profile.release]
opt-level = 3
lto = "thin"
codegen-units = 1
panic = "abort"
strip = true

# 5. Global dependency overrides
[patch.crates-io]
```

```
# Example: Point a buggy dependency to a local patched fork during debugging
# reqwest = { path = "../forks/reqwest" }
```

---

*User prompt: how to add a cargo package to root level cargo.toml file using cargo add*

Response: To add a dependency to the root `Cargo.toml` in a workspace, use the **`--workspace`** flag with `cargo add`.

### 1. Adding to `[workspace.dependencies]` (Virtual Workspace)

If your root `Cargo.toml` is a virtual manifest (it contains `[workspace]` and shared crates, but no `[package]` block), running a bare `cargo add` will fail with `error: cannot add dependency to virtual manifest`.

Use the `--workspace` flag from the root directory:

```
cargo add <package_name> --workspace
```

**Examples**

- Add a basic package:

  ```
  cargo add serde --workspace
  ```

- Add with specific features:

  ```
  cargo add tokio --workspace --features full
  ```

- Add a local path crate to workspace dependencies:

  ```
  cargo add core-types --workspace --path crates/core-types
  ```

This automatically writes the dependency into the `[workspace.dependencies]` table in your root `Cargo.toml`:

```
[workspace.dependencies]
serde = "1.0"
tokio = { version = "1.38", features = ["full"] }
core-types = { path = "crates/core-types" }
```

### 2. Consuming the Workspace Dependency in Member Crates

Once added to the root `[workspace.dependencies]`, navigate to any member crate or use `-p` to inherit it:

```
cargo add serde -p ingest-api
```

Cargo automatically detects that `serde` is defined in `[workspace.dependencies]` and configures the member's `Cargo.toml` as:

```
[dependencies]
serde = { workspace = true }
```

### 3. Adding to a Root Package (Non-Virtual Workspace)

If your root `Cargo.toml` defines both a `[package]` and a `[workspace]`:

- **Add to root's own `[dependencies]`:**

  ```
  cargo add <package_name>
  ```

- **Add to `[workspace.dependencies]`:**

  ```
  cargo add <package_name> --workspace
  ```

- **Target root package explicitly from any subfolder:**

  ```
  cargo add <package_name> --package <root_package_name>
  ```

---

*User prompt: cargo add anyhow --workspace error: unexpected argument '--workspace' found tip: to pass '--workspace' as a value, use '-- --workspace' Usage: cargo add [OPTIONS] <DEP>[@<VERSION>] ... cargo add [OPTIONS] --path <PATH> ... cargo add [OPTIONS] --git <URL> ...*

Response: `cargo add` does not have a `--workspace` flag—Cargo's built-in CLI currently does not support adding dependencies directly to the `[workspace.dependencies]` table of a virtual manifest.

In a virtual workspace (a root `Cargo.toml` with `[workspace]` but no `[package]`), `cargo add` only knows how to modify individual package manifests, not the workspace-level catalog.

### The Standard Workflow

#### 1. Add the dependency manually to the root `Cargo.toml`

Open your root `Cargo.toml` and place `anyhow` directly under `[workspace.dependencies]`:

```
[workspace.dependencies]
anyhow = "1.0"
```

#### 2. Add it to a member crate using `cargo add`

Once defined in the root manifest, Cargo will recognize it when you add it to any member crate. Run:

```
cargo add anyhow -p ingest-api
```

*(or run `cargo add anyhow` directly from inside `services/ingest-api/`)*

Cargo automatically detects that `anyhow` is defined in `[workspace.dependencies]` and configures `services/ingest-api/Cargo.toml` with the inherited syntax:

```
[dependencies]
anyhow.workspace = true
```

### Command-Line Workaround (Without an Editor)

If you prefer to append directly from your shell without opening an editor, you can append it directly to your root manifest:

```
# Append to the end of Cargo.toml (ensure [workspace.dependencies] is the last section)
echo 'anyhow = "1.0"' >> Cargo.toml
```

Then run `cargo check` to verify the workspace manifest resolves cleanly.

---

*User prompt: git add . warning: in the working copy of 'Cargo.lock', LF will be replaced by CRLF the next time Git touches it error: 'crates/core-types/' does not have a commit checked out fatal: adding files failed*

Response: This error happens because `cargo new` automatically initialized a separate, hidden `.git` folder inside `crates/core-types/` (and likely inside your other sub-crates).

When the root repository sees a nested `.git` folder, it treats that folder as a **Git submodule** instead of a normal directory, causing `git add .` to fail.

### Step 1: Remove the Nested `.git` Folders

Delete the hidden `.git` folders inside your subdirectories (keep only the root `.git` folder).

**If using Git Bash / macOS / Linux:**

```
# Delete all nested .git folders inside subdirectories
find . -mindepth 2 -name ".git" -type d -prune -exec rm -rf {} +
```

*Or explicitly delete them by folder:*

```
rm -rf crates/*/.git services/*/.git frontend/*/.git
```

**If using Windows PowerShell:**

```
Get-ChildItem -Path . -Filter .git -Recurse -Hidden -Directory | Where-Object { $_.FullName -ne (Get-Item .git).FullName
```

### Step 2: Clear Git's Index Cache

If Git cached `crates/core-types/` as a submodule entry, clear it from the index:

```
git rm --cached -r crates/core-types
```

*(If other crates gave the same warning, run `git rm --cached -r .` to reset the staged index).*

### Step 3: Fix the CRLF Line Ending Warning (Optional)

The warning `LF will be replaced by CRLF` occurs on Windows because Linux/macOS uses `LF` for line breaks, while Windows uses `CRLF`. Git is letting you know it will normalize them.

To silence this and let Git handle Windows line breaks automatically:

```
git config core.autocrlf true
```

### Step 4: Re-add and Commit

Now stage and commit your entire workspace clean:

```
git add .
git status
git commit -m "Initial workspace setup with crates, services, and frontend"
```

> **Pro-tip for future crates:** Whenever you create sub-crates inside an existing Cargo workspace, pass the `--vcs none` flag so Cargo doesn't initialize nested Git repos:
>
> ```
> cargo new --lib crates/new-crate --vcs none
> ```

---

*User prompt: git push -u origin main error: src refspec main does not match any error: failed to push some refs to 'https://github.com/vijaysarma1007/distributed-webhook-relay-and-real-time-event-debugger.git'*

Response: This error happens for one of two common reasons:

1. **You haven't committed any files yet** (Git cannot push an empty branch that has zero commits).
2. **Your local branch is named** `master` instead of `main` (the default in many Git installations).

### Step-by-Step Fix

Run the following commands in your project root:

#### 1. Stage and Commit Your Files

Make sure your changes are actually committed:

```
git add .
git commit -m "Initial commit: workspace structure and services"
```

*(If Git says `nothing to commit, working tree clean`, proceed to Step 2).*

#### 2. Rename Your Branch to `main`

Ensure your active branch is explicitly named `main`:

```
git branch -M main
```

#### 3. Push to GitHub

Now push and set the upstream tracking:

```
git push -u origin main
```

### If the Push is Rejected (`non-fast-forward` / remote contains work)

If you checked the box on GitHub to create a `README.md` or `.gitignore` when setting up the repository, the remote already has a commit that your local machine does not have.

Rebase against the remote and push:

```
git pull origin main --rebase
git push -u origin main
```

*(Alternatively, if your local code is the source of truth and the remote repository only has a dummy README, overwrite it with `git push -u origin main --force`).*

---

*User prompt: i have added all the member crate, from where should i begin*

Response: In a Rust workspace, always build **from the inside out** along your dependency graph: start with the shared domain types that have zero internal dependencies, move to the database layer, and then wire up your first runnable service.

### Phase 1: Shared Models in `crates/core-types`

Every service depends on shared request/response models and database entities. If these do not exist, none of your microservices will compile.

#### 1. Configure `crates/core-types/Cargo.toml`

```
[package]
name = "core-types"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { workspace = true }
serde_json = { workspace = true }
uuid = { workspace = true }
chrono = { workspace = true }
sqlx = { workspace = true, optional = true }

[features]
default = []
db = ["dep:sqlx"]
```

**2. Implement the Shared Types (`crates/core-types/src/lib.rs`)**

```
use chrono::{DateTime, Utc};
use serde::{Deserialize, Serialize};
use uuid::Uuid;

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
#[serde(rename_all = "SCREAMING_SNAKE_CASE")]
pub enum DeliveryStatus {
    Pending,
    Delivered,
    Failed,
    DeadLetter,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct IngestWebhookRequest {
    pub destination_url: String,
    pub payload: serde_json::Value,
    pub secret_key: String,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[cfg_attr(feature = "db", derive(sqlx::FromRow))]
pub struct OutboxEvent {
    pub id: Uuid,
    pub destination_url: String,
    pub payload: serde_json::Value,
    pub secret_key: String,
    pub status: String,
    pub attempt_count: i32,
    pub max_attempts: i32,
    pub next_retry_at: DateTime<Utc>,
    pub created_at: DateTime<Utc>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[cfg_attr(feature = "db", derive(sqlx::FromRow))]
pub struct DeliveryAttempt {
    pub id: Uuid,
    pub event_id: Uuid,
    pub http_status: Option<i32>,
    pub response_body: Option<String>,
    pub latency_ms: i32,
    pub error_message: Option<String>,
    pub attempted_at: DateTime<Utc>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct LiveEventNotification {
    pub attempt_id: Uuid,
    pub event_id: Uuid,
    pub http_status: Option<i32>,
    pub latency_ms: i32,
    pub destination_url: String,
    pub attempted_at: DateTime<Utc>,
}
```

Verify it compiles:

```
cargo check -p core-types
```

### Phase 2: Local Database & `crates/db-store`

Before writing service logic, get PostgreSQL running and managed via SQLx migrations.

#### 1. Spin Up Local PostgreSQL

Create a `docker-compose.yml` in your project root:

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: relay-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: webhook_relay
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Start the container:

```
docker compose up -d
```

#### 2. Create the Database Schema Migration

Create directory `crates/db-store/migrations/` and add file `20260101000000_init.sql`:

```
CREATE TABLE outbox_events (
    id UUID PRIMARY KEY,
    destination_url TEXT NOT NULL,
    payload JSONB NOT NULL,
    secret_key TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'PENDING',
    attempt_count INT NOT NULL DEFAULT 0,
    max_attempts INT NOT NULL DEFAULT 5,
    next_retry_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_outbox_queue ON outbox_events (next_retry_at ASC)
WHERE status = 'PENDING';

CREATE TABLE delivery_attempts (
    id UUID PRIMARY KEY,
    event_id UUID NOT NULL REFERENCES outbox_events(id) ON DELETE CASCADE,
    http_status INT,
    response_body TEXT,
    latency_ms INT NOT NULL,
    error_message TEXT,
    attempted_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION notify_delivery_attempt()
RETURNS TRIGGER AS $$
DECLARE
    target_url TEXT;
BEGIN
    SELECT destination_url INTO target_url FROM outbox_events WHERE id = NEW.event_id;

    PERFORM pg_notify(
        'webhook_stream',
        json_build_object(
            'attempt_id', NEW.id,
            'event_id', NEW.event_id,
            'http_status', NEW.http_status,
```

```
            'latency_ms', NEW.latency_ms,
            'destination_url', target_url,
            'attempted_at', NEW.attempted_at
        )::text
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_delivery_attempt
AFTER INSERT ON delivery_attempts
FOR EACH ROW EXECUTE FUNCTION notify_delivery_attempt();
```

Apply the migration:

```
export DATABASE_URL="postgres://postgres:password@localhost:5432/webhook_relay"
sqlx database create
sqlx migrate run --source crates/db-store/migrations
```

**3. Implement the Store (`crates/db-store/src/lib.rs`)**

Configure `crates/db-store/Cargo.toml`:

```
[package]
name = "db-store"
version = "0.1.0"
edition = "2021"

[dependencies]
core-types = { workspace = true, features = ["db"] }
sqlx = { workspace = true }
uuid = { workspace = true }
chrono = { workspace = true }
thiserror = { workspace = true }
```

Implement `crates/db-store/src/lib.rs`:

```
use core_types::{IngestWebhookRequest, OutboxEvent};
use sqlx::{PgPool, Result};
use uuid::Uuid;

#[derive(Clone)]
pub struct DbStore {
    pool: PgPool,
}

impl DbStore {
    pub fn new(pool: PgPool) -> Self {
        Self { pool }
    }

    pub fn pool(&self) -> &PgPool {
        &self.pool
    }

    pub async fn insert_outbox_event(&self, req: IngestWebhookRequest) -> Result<Uuid> {
        let event_id = Uuid::now_v7();

        sqlx::query!(
            r#"
            INSERT INTO outbox_events (id, destination_url, payload, secret_key)
            VALUES ($1, $2, $3, $4)
            "#,
            event_id,
            req.destination_url,
            req.payload,
            req.secret_key
        )
        .execute(&self.pool)
        .await?;

        Ok(event_id)
    }

    pub async fn fetch_pending_events(&self, batch_size: i64) -> Result<Vec<OutboxEvent>> {
```

```
        sqlx::query_as!(
            OutboxEvent,
            r#"
            SELECT
                id, destination_url, payload, secret_key,
                status, attempt_count, max_attempts, next_retry_at, created_at
            FROM outbox_events
            WHERE status = 'PENDING' AND next_retry_at <= NOW()
            ORDER BY next_retry_at ASC
            LIMIT $1
            FOR UPDATE SKIP LOCKED
            "#,
            batch_size
        )
        .fetch_all(&self.pool)
        .await
    }
}
```

### Phase 3: The First Runnable Microservice (`services/ingest-api`)

Building `ingest-api` gives you an immediate working feedback loop: an HTTP endpoint where you can send POST requests with `curl` and verify they appear in PostgreSQL.

**1. Configure `services/ingest-api/Cargo.toml`**

```
[package]
name = "ingest-api"
version = "0.1.0"
edition = "2021"

[dependencies]
core-types = { workspace = true }
db-store = { workspace = true }
tokio = { workspace = true }
axum = { workspace = true }
tower-http = { workspace = true }
sqlx = { workspace = true }
tracing = { workspace = true }
tracing-subscriber = { workspace = true }
dotenvy = { workspace = true }
serde_json = { workspace = true }
```

**2. Implement the API Server (`services/ingest-api/src/main.rs`)**

```
use axum::{
    extract::State,
    http::StatusCode,
    response::IntoResponse,
    routing::post,
    Json, Router,
};
use core_types::IngestWebhookRequest;
use db-store::DbStore;
use sqlx::postgres::PgPoolOptions;
use std::net::SocketAddr;
use tracing::info;

#[derive(Clone)]
struct AppState {
    store: DbStore,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    tracing_subscriber::fmt::init();
    dotenvy::dotenv().ok();

    let database_url = std::env::var("DATABASE_URL")
        .unwrap_or_else(|_| "postgres://postgres:password@localhost:5432/webhook_relay".into());

    let pool = PgPoolOptions::new()
        .max_connections(20)
```

```
        .connect(&database_url)
        .await?;

    let state = AppState {
        store: DbStore::new(pool),
    };

    let app = Router::new()
        .route("/api/v1/events", post(handle_ingest))
        .with_state(state);

    let addr = SocketAddr::from(([0, 0, 0, 0], 3000));
    info!("ingest-api listening on {}", addr);

    let listener = tokio::net::TcpListener::bind(addr).await?;
    axum::serve(listener, app).await?;

    Ok(())
}

async fn handle_ingest(
    State(state): State<AppState>,
    Json(payload): Json<IngestWebhookRequest>,
) -> impl IntoResponse {
    match state.store.insert_outbox_event(payload).await {
        Ok(event_id) => (
            StatusCode::ACCEPTED,
            Json(serde_json::json!({
                "status": "queued",
                "event_id": event_id
            })),
        ),
        Err(err) => (
            StatusCode::INTERNAL_SERVER_ERROR,
            Json(serde_json::json!({
                "error": err.to_string()
            })),
        ),
    }
}
```

**3. Test the First Milestone**

Run the service:

```
cargo run -p ingest-api
```

Send a test webhook:

```
curl -X POST http://localhost:3000/api/v1/events \
  -H "Content-Type: application/json" \
  -d '{
    "destination_url": "https://httpbin.org/post",
    "payload": { "event": "order.completed", "amount": 9900 },
    "secret_key": "supersecretkey"
  }'
```

Verify that the record was created in the database:

```
docker exec -it relay-postgres psql -U postgres -d webhook_relay -c "SELECT id, destination_url, status FROM outbox_even
```

Once this pipeline runs end-to-end, you can proceed to writing the `dispatch-worker` polling loop to deliver those queued events.

---

*User prompt: what is the purpose of core-types*

Response: In this architecture, **`core-types` serves as the single source of truth and shared contract** for all data models, transfer objects (DTOs), and domain entities across your entire workspace.

Without `core-types`, every microservice (`ingest-api`, `dispatch-worker`, `stream-bridge`) and library crate (`db-store`) would have to redefine its own duplicate versions of events, delivery records, and status enums.

## Key Purposes & Problems It Solves

**1. Defining the Contract Between Microservices**

In a distributed system, services talk to each other via network payloads or the database. `core-types` guarantees that:

- The format `ingest-api` writes into the database matches the format `dispatch-worker` reads.
- The notification payload emitted by PostgreSQL is deserialized by `stream-bridge` with the exact same fields expected by clients.
- If a field name or type changes, **the Rust compiler immediately catches breaking changes across all services** at build time, preventing runtime JSON mismatch errors.

**2. Eliminating Code Duplication (DRY)**

Instead of copy-pasting structs into three different services, you define models once. When you add a new property (like a `headers` field to an event), you update it in one place, and all consuming crates inherit the update.

**3. Preventing Circular Dependencies (The "Leaf Node" Pattern)**

In Cargo workspaces, circular dependencies are forbidden (e.g., crate A cannot depend on crate B if crate B depends on crate A).

- `core-types` sits at the very bottom of the dependency graph as a **leaf crate**.
- It only depends on low-level primitives (`serde`, `uuid`, `chrono`).
- It **does not** depend on heavy web frameworks (`axum`), database engines (`sqlx`), or HTTP clients (`reqwest`).
- Because it is lightweight, any crate or service in the workspace can import it freely without pulling in unnecessary runtime bloat.

**4. Shared Serialization and Validation Rules**

Serialization logic (such as casing conventions like `#[serde(rename_all = "SCREAMING_SNAKE_CASE")]` or custom timestamp parsers) is declared once in `core-types`. This ensures that every service serializes enums and timestamps identically over the wire.

### What Lives Inside `core-types` in this Project

| Type | Purpose | Consumers |
|---|---|---|
| `IngestWebhookRequest` | The shape of the incoming JSON payload sent by webhook producers. | `ingest-api` |
| `OutboxEvent` | The representation of a queued job waiting in the database outbox. | `db-store`, `dispatch-worker` |
| `DeliveryAttempt` | The recorded result of an HTTP delivery attempt (status code, latency, error). | `db-store`, `dispatch-worker`, `stream-bridge` |
| `DeliveryStatus` | Enum for state transitions (`Pending`, `Delivered`, `Failed`, `DeadLetter`). | `db-store`, `dispatch-worker` |
| `LiveEventNotification` | Streamlined telemetry event sent over SSE/WebSockets to the UI. | `stream-bridge`, `debugger-ui` |

---

*User prompt: what is the purpose of stream-bridge bin*

Response: The **`stream-bridge`** binary is a dedicated telemetry gateway whose sole job is to convert internal PostgreSQL events into real-time, push-based streams (Server-Sent Events or WebSockets) for the frontend debugger UI.

It connects your internal database state to external browser screens without polling.

### The Problem It Solves

Without `stream-bridge`, your frontend UI would have to use **polling**—asking the server every second:

> *"Are there new events? Are there new events? Are there new events?"*

If 100 developers or support reps open the dashboard, this generates thousands of repetitive `SELECT` queries every minute, hammering the database with empty queries.

Instead of polling, `stream-bridge` makes the system **event-driven and push-based**: the frontend opens a single, persistent HTTP connection, and `stream-bridge` pushes new events to the browser the millisecond they occur.

### How It Operates Under the Hood

```
[ PostgreSQL ]
      |
```

```
      │ 1. Trigger fires: pg_notify('webhook_activity', json_payload)
      ▼
[ services/stream-bridge ]
      │
      │ 2. sqlx::postgres::PgListener catches notification
      │ 3. Pushes into tokio::sync::broadcast::channel
      ▼
[ Connected Clients ]
      ├──► Browser 1 (Leptos UI)  <-- SSE Stream (text/event-stream)
      ├──► Browser 2 (Leptos UI)  <-- SSE Stream
      └──► Browser N (CLI / Logs) <-- SSE Stream
```

1. **Listens to Postgres:** When `dispatch-worker` finishes an HTTP attempt, it writes a row to `delivery_attempts`. A Postgres trigger automatically runs:

   ```
   PERFORM pg_notify('webhook_stream', '{"attempt_id": "...", "status": 200, "latency_ms": 42}');
   ```

2. **Listens via `PgListener`:** `stream-bridge` keeps an open listener task using `sqlx::postgres::PgListener`. It sleeps with zero CPU usage until a notification packet arrives from Postgres.
3. **In-Memory Broadcast:** When an event arrives, `stream-bridge` decodes it and sends it into a `tokio::sync::broadcast` queue in memory.
4. **SSE Endpoint (`GET /events/stream`):** Any browser displaying the Leptos UI connects to this endpoint. The service fans out incoming messages from the broadcast channel to all connected browser streams simultaneously.

### Why It Is an Independent Microservice

It might seem tempting to put this logic inside `ingest-api` or `dispatch-worker`, but keeping it isolated provides three critical architectural advantages:

- **Long-Lived vs. Short-Lived Connections:** `ingest-api` handles rapid, stateless request-and-response traffic (lasting 5 to 20ms). In contrast, SSE streams are long-lived connections that remain open for minutes or hours. Keeping them in a separate process prevents long-lived UI connections from starving or exhausting the HTTP socket pools of your critical ingestion gateway.
- **Failure Isolation:** If the `dispatch-worker` crashes or runs out of memory under high outbound network pressure, the monitoring bridge remains completely intact. You can still watch reconnect attempts, error states, and telemetry.
- **Horizontal Scaling Differences:**
  - `ingest-api` scales based on how many incoming webhooks third parties send.
  - `dispatch-worker` scales based on how deep the outbox queue is.
  - `stream-bridge` scales based on how many internal operators/developers are viewing the dashboard. Separating them lets you allocate CPU and RAM independently to each workload.

---

*User prompt: db = ["dep:sqlx"] , what's dep:sqlx why it is needed*

Response: `dep:sqlx` is Cargo's explicit syntax for enabling an optional dependency inside a feature without exposing the dependency's name as a public crate feature.

It was introduced in Rust 1.60 under the "namespaced features" RFC to solve two main problems: public API leakage and dependency coupling.

### 1. The Default (Old) Behavior: Leaking Dependency Names

If you declare an optional dependency like this:

```
[dependencies]
sqlx = { workspace = true, optional = true }

[features]
db = ["sqlx"]
```

Cargo automatically creates an implicit public feature named `sqlx`. This means users of your crate can enable the database support in two different ways:

```
# Both work, even though you intended them to only use "db"
core-types = { version = "0.1", features = ["db"] }
core-types = { version = "0.1", features = ["sqlx"] }
```

This creates problems:

- **Leaked Implementation Details:** Your crate's public API is tied directly to the name of the third-party crate.

- **Breaking Changes on Refactors:** If you later switch from `sqlx` to another database library (like `tokio-postgres` or `diesel`), changing the dependency breaks code for any consumer who enabled `features = ["sqlx"]`.

### 2. The `dep:` Syntax: Hiding the Dependency

When you prefix the dependency with `dep:` in a feature definition:

```
[dependencies]
sqlx = { workspace = true, optional = true }

[features]
default = []
db = ["dep:sqlx"]
```

Cargo stops creating the implicit `sqlx` feature.

- The `sqlx` crate is **only** compiled when the `db` feature is activated.
- Consumers can **only** write `features = ["db"]`.
- Writing `features = ["sqlx"]` results in a compilation error: `Package does not have the feature 'sqlx'`.

### Why It Is Used in `crates/core-types`

In `core-types`, we want the crate to remain as lightweight as possible. Services that only parse JSON (like a CLI or frontend) shouldn't be forced to compile SQLx.

By using `db = ["dep:sqlx"]`:

1. **Clean Feature Gate:** Consumers enable database derives by declaring `features = ["db"]`.
2. **Conditional Derives:** The code in `crates/core-types/src/lib.rs` uses:

   ```
   #[derive(Debug, Clone, Serialize, Deserialize)]
   #[cfg_attr(feature = "db", derive(sqlx::FromRow))]
   pub struct OutboxEvent { ... }
   ```

   When `db` is inactive, `sqlx` is neither downloaded nor compiled, and the struct remains a plain data structure. When `db` is active, Cargo brings in `sqlx` to provide the `FromRow` derive macro.

---

*User prompt: pub enum DeliveryStatus { Pending, Delivered, Failed, DeadLetter, } whats deadlock here?*

Response: There is likely a mix-up between two similar-sounding terms: **`DeadLetter`** (which is written in your enum) and a concurrency **`Deadlock`**.

They represent completely different concepts in system design:

### 1. What `DeadLetter` Means in this Enum

The variant in your code is **`DeadLetter`**, not "deadlock."

The term comes from the postal service: a **"Dead Letter Office"** is where postal workers send undeliverable letters when the destination address is invalid or the recipient no longer lives there.

In messaging systems and webhook engines, **`DeadLetter`** means **permanent, terminal failure**:

- **`Pending`**: Waiting to be picked up and dispatched.
- **`Delivered`**: Successfully reached the destination (`2xx OK`).
- **`Failed`**: A temporary failure (e.g., network hiccup, HTTP `503`, or timeout). The worker will retry it later.
- **`DeadLetter`**: The delivery failed repeatedly past the maximum retry limit (e.g., 5 attempts over 24 hours) or failed with a non-retryable error (e.g., `404 Not Found` or `410 Gone`).

When a record is marked `DeadLetter`:

1. The dispatch worker **stops retrying** it, saving server CPU and network bandwidth.
2. The event is quarantined so an operator can inspect the payload in the **Debugger UI** to see why the client rejected it, fix the target URL, or click "Replay".

### 2. What a Concurrency `Deadlock` Actually Is

A **deadlock** is not an enum status—it is a runtime concurrency bug where two processes freeze permanently because each is waiting for a lock held by the other.

**Example of a Database Deadlock:**

1. **Worker 1** locks Event A and wants to update Event B.
2. **Worker 2** locks Event B and wants to update Event A.
3. Neither worker can proceed, so the database detects a deadlock and kills one of the transactions with an error (`40P01: deadlock detected`).

**How this Architecture Prevents Deadlocks:**

The webhook engine avoids deadlocks during queue processing by using:

```
SELECT * FROM outbox_events
WHERE status = 'PENDING' AND next_retry_at <= NOW()
ORDER BY next_retry_at ASC
LIMIT 50
FOR UPDATE SKIP LOCKED;
```

`SKIP LOCKED` instructs PostgreSQL to bypass any row another worker is currently updating, ensuring workers never block each other or cause a deadlock.

---

*User prompt: #[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)] why partialEq, clone and Eq is required*

Response: In Rust, basic capabilities like comparing values or making copies are not built into custom types by default. You must explicitly opt into them by deriving or implementing specific traits.

## 1. `PartialEq`: Enables Comparison (`==` and `!=`)

By default, Rust does not know how to check if two instances of your type are equal. Deriving `PartialEq` implements the `==` and `!=` operators.

**Without `PartialEq`:**

```
// ❌ This will not compile!
if event.status == DeliveryStatus::Pending {
    // Error: binary operation `==` cannot be applied to type `DeliveryStatus`
}
```

**What It Enables:**

- **Conditional logic:** `if status == DeliveryStatus::Pending { ... }`
- **Unit testing:** `assert_eq!(job.status, DeliveryStatus::Delivered);`
- **Iterator filtering:** `events.iter().filter(|e| e.status == DeliveryStatus::Failed)`

## 2. `Eq`: Guarantees Full Equivalence

`Eq` is a marker trait that builds on top of `PartialEq`. It does not add new methods, but it signals to the Rust compiler that equality is **reflexive**: for every value `x`, **`x == x` is always true.**

**Why `PartialEq` is not enough for `Eq`:**

In standard programming, floating-point numbers like `f32::NAN` break reflexivity because `NAN == NAN` is `false`. Therefore, `f32` implements `PartialEq`, but **not** `Eq`.

Because an enum like `DeliveryStatus` consists of discrete, well-defined variants, reflexivity is guaranteed:

- `DeliveryStatus::Pending == DeliveryStatus::Pending` is always `true`.

**What `Eq` Enables:**

- **Hash map & Set keys:** To use a type as a key in a `HashSet<T>` or `HashMap<K, V>`, Rust requires both `Hash` and `Eq`.
- **Pattern matching optimizations:** The compiler can make stronger optimization assumptions in match tables and sorting algorithms when a type is guaranteed to be fully reflexive.

## 3. `Clone`: Enables Explicit Duplication (`.clone()`)

Rust uses strict ownership semantics: assigning a variable or passing it into a function **moves** ownership by default. Deriving `Clone` allows you to create an independent, duplicate copy of the value using `.clone()`.

**Where It Is Essential:**

- **Tokio Tasks and Channels:** When sending events across async tasks via `tokio::sync::mpsc` or `tokio::sync::broadcast`, the receiver needs its own owned copy if the original must stay in scope.
- **Leptos Signals (Frontend):** Reactive frameworks frequently clone state when passing data into closures or view templates:

  ```
  let (status, set_status) = create_signal(cx, initial_status);
  // Reading or passing status into reactive closures requires Clone
  ```

- **Database Updates:** When constructing log entries or audit rows while retaining the original struct in memory.

### Bonus: Can this type also derive `Copy`?

For a simple C-like enum with no heap allocations (no strings, vectors, or pointers), you can also derive `Copy`:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum DeliveryStatus {
    Pending,
    Delivered,
    Failed,
    DeadLetter,
}
```

- `Copy` makes duplication **implicit via bitwise copy** (just like primitive integers or booleans).
- Passing `status` into a function automatically copies it without consuming ownership or requiring an explicit `.clone()` call.

---

*User prompt: in #[cfg_attr(feature = "db", derive(sqlx::FromRow))] derive(sqlx::FromRow) what it does ?*

Response: `derive(sqlx::FromRow)` automatically generates the code needed to **convert a raw database row into your Rust struct.**

When SQLx runs a query against PostgreSQL, the database returns raw binary or text rows (`PgRow`). Deriving `FromRow` tells SQLx how to read each column by name and instantiate your struct without writing repetitive mapping code by hand.

### The Problem Without `FromRow` (Manual Extraction)

If you do not derive `FromRow`, you have to extract every column from the database row manually:

```
use sqlx::{Row, postgres::PgRow};

// Without FromRow, you must manually fetch each column
let row: PgRow = sqlx::query("SELECT id, destination_url, attempt_count FROM outbox_events")
    .fetch_one(&pool)
    .await?;

let event = OutboxEvent {
    id: row.try_get("id")?,
    destination_url: row.try_get("destination_url")?,
    attempt_count: row.try_get("attempt_count")?,
    // ... repeating this for every single field
};
```

This approach is error-prone: a single typo in a column string fails at runtime, and adding a field requires updating queries and manual mappings across the codebase.

### The Solution With `FromRow` (Automated Mapping)

When you add `#[derive(sqlx::FromRow)]`, SQLx implements the `sqlx::FromRow` trait at compile time:

```
#[derive(Debug, sqlx::FromRow)]
pub struct OutboxEvent {
    pub id: Uuid,
    pub destination_url: String,
    pub attempt_count: i32,
}
```

Now you can use SQLx's typed query APIs like `query_as`:

```
// SQLx inspects column names and constructs OutboxEvent automatically
let events: Vec<OutboxEvent> = sqlx::query_as::<_, OutboxEvent>(
    "SELECT id, destination_url, attempt_count FROM outbox_events"
)
.fetch_all(&pool)
.await?;
```

SQLx looks at each field name in your struct (`id`, `destination_url`, `attempt_count`), looks for a matching column in the SQL result set, deserializes the database value into the corresponding Rust type, and constructs the struct.

### How `#[cfg_attr(feature = "db", ...)]` Fits In

In your shared `core-types` crate:

```
#[cfg_attr(feature = "db", derive(sqlx::FromRow))]
pub struct OutboxEvent { ... }
```

`cfg_attr` stands for **conditional configuration attribute**:

- **If `feature = "db"` is ON** (e.g., in `db-store` or backend workers): Rust applies `#[derive(sqlx::FromRow)]`, allowing direct database mapping.
- **If `feature = "db"` is OFF** (e.g., in `frontend/debugger-ui` or WASM code): the attribute is completely ignored. The frontend can use `OutboxEvent` for JSON parsing without pulling the heavy `sqlx` engine into the browser bundle.

---

*User prompt: #[derive(Debug, Clone, Serialize, Deserialize)]pub struct LiveEventNotification { pub attempt_id: Uuid, pub event_id: Uuid, pub http_status: Option<i32>, pub latency_ms: i32, pub destination_url: String, pub attempted_at: DateTime<Utc>, } explain why it is required in this project and where exactly to use it ?*

Response: `LiveEventNotification` is a specialized **Data Transfer Object (DTO)** designed for low-overhead, real-time telemetry streaming from PostgreSQL to your Leptos frontend.

While database tables store comprehensive records (raw request payloads, response bodies, secret keys, headers), this struct represents the minimal metadata needed to display a live event row on the debugger dashboard.

### Why It Is Required

#### 1. Bandwidth Optimization (Preventing Network Flooding)

The full `outbox_events` and `delivery_attempts` tables contain heavy columns such as `payload` (which could be a 500 KB JSON document) and `response_body` (potentially large HTML error pages).

- If your system dispatches 1,000 webhooks per second, broadcasting full database rows over Server-Sent Events (SSE) would require sending hundreds of megabytes of redundant data per second to every connected browser.
- `LiveEventNotification` contains only lightweight primitives (UUIDs, integers, timestamps, a short URL string), keeping the telemetry stream under **~200 bytes per event**.

#### 2. Security & Secret Redaction

Your `outbox_events` table stores `secret_key`, which is used to compute HMAC-SHA256 signatures. Broadcasting full database records across WebSockets or SSE channels risks leaking secret keys or sensitive customer PII to browser sessions. `LiveEventNotification` acts as a security boundary, excluding credentials entirely.

#### 3. Decoupling the UI Feed from the Database Schema

The live ticker on the dashboard only needs to show status indicators (e.g., 🟢 `200 OK`, 🔴 `502 Bad Gateway`), elapsed latency (e.g., `35ms`), destination, and timestamps. Detailed data (like full headers and bodies) can be fetched on demand via a separate REST endpoint only when a developer clicks on an individual row.

### Where Exactly It Is Used in the Codebase

This struct forms the end-to-end data bridge across three distinct parts of the system:

```
[ PostgreSQL Trigger ]
        │ (Emits JSON via pg_notify)
        ▼
[ services/stream-bridge ]
        │ (Deserializes into LiveEventNotification)
        │ (Broadcasts over Tokio channel to SSE endpoint)
        ▼
[ frontend/debugger-ui (Leptos) ]
        │ (Deserializes SSE message and renders UI row)
```

#### 1. In `services/stream-bridge`: Ingestion from PostgreSQL

The PostgreSQL trigger `notify_delivery_attempt` emits a JSON payload on the channel `webhook_stream`. The `stream-bridge` service listens to this channel and deserializes the raw string directly into `LiveEventNotification`:

```rust
// services/stream-bridge/src/main.rs
use core_types::LiveEventNotification;
use sqlx::postgres::PgListener;
use tokio::sync::broadcast;

async fn listen_to_pg_stream(
    mut listener: PgListener,
    tx: broadcast::Sender<LiveEventNotification>,
) -> Result<(), Box<dyn std::error::Error>> {
    listener.listen("webhook_stream").await?;

    while let Ok(notification) = listener.recv().await {
        // Deserialize the database trigger JSON into the typed struct
        if let Ok(event) = serde_json::from_str::<LiveEventNotification>(notification.payload()) {
            // Broadcast to all active client connections
            let _ = tx.send(event);
        }
    }

    Ok(())
}
```

**2. In `services/stream-bridge`: Streaming Over Server-Sent Events (SSE)**

When the browser connects to `GET /events/stream`, the Axum handler subscribes to the broadcast channel, serializes `LiveEventNotification` into an SSE data frame, and streams it to the client:

```rust
// services/stream-bridge/src/sse.rs
use axum::response::sse::{Event, KeepAlive, Sse};
use core_types::LiveEventNotification;
use futures::stream::Stream;
use tokio::sync::broadcast;
use tokio_stream::wrappers::BroadcastStream;
use tokio_stream::StreamExt;

pub async fn sse_handler(
    tx: broadcast::Sender<LiveEventNotification>,
) -> Sse<impl Stream<Item = Result<Event, axum::Error>>> {
    let rx = tx.subscribe();
    let stream = BroadcastStream::new(rx).filter_map(|msg| match msg {
        Ok(notification) => {
            let json = serde_json::to_string(&notification).unwrap_or_default();
            Some(Ok(Event::default().data(json)))
        }
        Err(_) => None,
    });

    Sse::new(stream).keep_alive(KeepAlive::default())
}
```

**3. In `frontend/debugger-ui`: Updating the Leptos Reactive View**

Inside your Leptos WASM app, the `LiveEventNotification` struct represents an incoming item in the live feed table:

```rust
// frontend/debugger-ui/src/app.rs
use core_types::LiveEventNotification;
use leptos::*;

#[component]
pub fn LiveFeedTable() -> impl IntoView {
    let (events, set_events) = create_signal(Vec::<LiveEventNotification>::new());

    // Connect to SSE stream on component mount
    create_effect(move |_| {
        use gloo_net::eventsource::futures::EventSource;
        use futures::StreamExt;

        let mut es = EventSource::new("http://localhost:3001/events/stream")
            .expect("Failed to connect to SSE");

        spawn_local(async move {
            while let Some(Ok((_, msg))) = es.next().await {
                if let Ok(raw) = msg.data().as_string() {
                    if let Ok(item) = serde_json::from_str::<LiveEventNotification>(&raw) {
```

```
                        set_events.update(|list| {
                            // Prepend latest delivery attempt to top of table
                            list.insert(0, item);
                            list.truncate(100); // Retain latest 100 entries
                        });
                    }
                }
            }
        });
    });

    view! {
        <table class="w-full text-left text-sm text-slate-300">
            <tbody>
                <For
                    each=move || events.get()
                    key=|item| item.attempt_id
                    children=move |item| {
                        let status_color = match item.http_status {
                            Some(200..=299) => "text-emerald-400 bg-emerald-950/50",
                            Some(400..=499) => "text-amber-400 bg-amber-950/50",
                            _ => "text-rose-400 bg-rose-950/50",
                        };

                        view! {
                            <tr class="border-b border-slate-800 hover:bg-slate-800/40">
                                <td class="py-2 px-3 font-mono text-xs">{item.event_id.to_string()}</td>
                                <td class="py-2 px-3">{item.destination_url}</td>
                                <td class="py-2 px-3">
                                    <span class={format!("px-2 py-0.5 rounded text-xs font-semibold {}", status_color)}>
                                        {item.http_status.map_or("ERR".into(), |s| s.to_string())}
                                    </span>
                                </td>
                                <td class="py-2 px-3 font-mono text-xs">{format!("{}ms", item.latency_ms)}</td>
                                <td class="py-2 px-3 text-xs text-slate-500">{item.attempted_at.to_rfc3339()}</td>
                            </tr>
                        }
                    }
                />
            </tbody>
        </table>
    }
}
```

### Comparison Summary

| Attribute | `DeliveryAttempt` (Database Row) | `LiveEventNotification` (Stream DTO) |
|---|---|---|
| **Location** | Stored in PostgreSQL disk partitions | Transient; lives only in memory and in-flight SSE streams |
| **Payload Size** | Heavy (includes `response_body` and error strings) | Ultra-light (~200 bytes) |
| **Destination URL** | Not stored (relies on SQL join to `outbox_events`) | Inlined directly to avoid frontend lookup queries |
| **Usage** | Audit trail, debugging deep-dives, manual inspection | Live ticker updates, UI status badges, latency charts |

---

*User prompt: how to calclute the latency_ms inside live event notifcation*

Response: In the dispatch engine, **`latency_ms` is measured using Rust's monotonic clock (`std::time::Instant`)** directly inside `services/dispatch-worker` before and after sending the outbound HTTP request. It is then saved into the `delivery_attempts` table, where the PostgreSQL trigger passes it along to `LiveEventNotification`.

### 1. Measuring Round-Trip Time in `dispatch-worker`

Always use `std::time::Instant` (a monotonic clock) rather than system wall-clock time (`Utc::now()`). Wall-clock time can jump backwards or forwards due to NTP sync adjustments, which can result in inaccurate or negative latency calculations.

```
// services/dispatch-worker/src/dispatcher.rs
use std::time::Instant;
use reqwest::Client;
use uuid::Uuid;
```

```rust
use core_types::OutboxEvent;

pub async fn dispatch_webhook(
    client: &Client,
    event: &OutboxEvent,
    signed_payload: String,
) -> (Option<i32>, i32, Option<String>, Option<String>) {
    // 1. Start the monotonic timer immediately before the network call
    let start_time = Instant::now();

    // 2. Execute the outbound HTTP request with a strict timeout
    let response_result = client
        .post(&event.destination_url)
        .header("Content-Type", "application/json")
        .header("X-Signature", signed_payload)
        .body(event.payload.to_string())
        .timeout(std::time::Duration::from_secs(10))
        .send()
        .await;

    // 3. Stop the timer immediately after response headers arrive (or on network failure)
    let elapsed = start_time.elapsed();

    // Convert to milliseconds and cap to i32::MAX to prevent overflow
    let latency_ms = elapsed.as_millis().min(i32::MAX as u128) as i32;

    // 4. Extract status code and body / error details
    match response_result {
        Ok(resp) => {
            let status = resp.status().as_u16() as i32;
            let body = resp.text().await.ok();
            (Some(status), latency_ms, body, None)
        }
        Err(err) => {
            let status = err.status().map(|s| s.as_u16() as i32);
            let error_msg = Some(err.to_string());
            (status, latency_ms, None, error_msg)
        }
    }
}
```

### 2. Saving `latency_ms` to PostgreSQL (`crates/db-store`)

Once measured, pass `latency_ms` to your database repository query to insert the attempt record:

```rust
// crates/db-store/src/lib.rs
pub async fn record_attempt(
    &self,
    event_id: Uuid,
    http_status: Option<i32>,
    latency_ms: i32,
    response_body: Option<String>,
    error_message: Option<String>,
) -> Result<Uuid, sqlx::Error> {
    let attempt_id = Uuid::now_v7();

    sqlx::query!(
        r#"
        INSERT INTO delivery_attempts (
            id, event_id, http_status, latency_ms, response_body, error_message, attempted_at
        )
        VALUES ($1, $2, $3, $4, $5, $6, NOW())
        "#,
        attempt_id,
        event_id,
        http_status,
        latency_ms,
        response_body,
        error_message
    )
    .execute(&self.pool)
    .await?;
```

```
    Ok(attempt_id)
}
```

### 3. Automatic Broadcast via PostgreSQL Trigger

Because the table schema has a trigger on `delivery_attempts`, PostgreSQL automatically takes the newly inserted `NEW.latency_ms` value and packages it into the notification JSON:

```
-- Trigger automatically packages NEW.latency_ms
PERFORM pg_notify(
    'webhook_stream',
    json_build_object(
        'attempt_id', NEW.id,
        'event_id', NEW.event_id,
        'http_status', NEW.http_status,
        'latency_ms', NEW.latency_ms,       -- <--- Forwarded here
        'destination_url', target_url,
        'attempted_at', NEW.attempted_at
    )::text
);
```

When `stream-bridge` receives this raw JSON string via `PgListener`, `serde_json::from_str::<LiveEventNotification>(&payload)` automatically maps the key `"latency_ms"` into `pub latency_ms: i32`.

### Summary of Best Practices

- **Always include failures:** If an endpoint times out after 10 seconds, `latency_ms` should accurately reflect `10000ms`, not `0ms`.
- **Measure before body streaming:** For minimal overhead, calculate latency as soon as the HTTP headers are received. If tracking full response transfer time is required, calculate `start_time.elapsed()` after `.bytes()` or `.text()` resolves.
- **Monotonic safety:** Using `Instant::now()` guarantees the value will never be negative even during leap seconds or Daylight Saving adjustments.

---

*User prompt: why using reqwest instead axum*

Response: **`axum` and `reqwest` perform opposite jobs: `axum` is an HTTP server (receives incoming requests), while `reqwest` is an HTTP client (sends outgoing requests).**

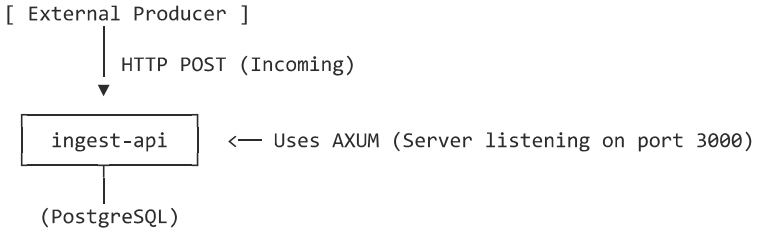
They are not alternatives to one another; rather, they work together in this architecture.

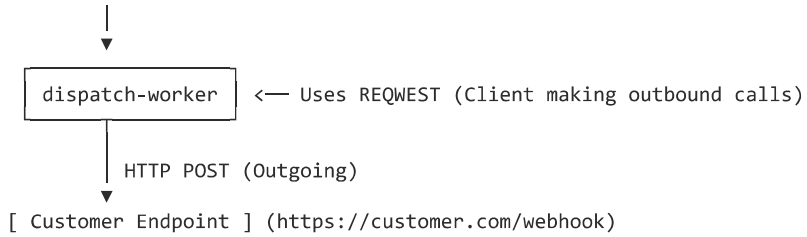
### The Fundamental Difference

| Feature | `axum` | `reqwest` |
|---|---|---|
| **Role** | **HTTP Server** | **HTTP Client** |
| **Direction** | **Inbound** (Listens for traffic) | **Outbound** (Initiates traffic) |
| **Action** | Opens a TCP port (`0.0.0.0:3000`), waits for requests, and sends responses back. | Connects to an external URL (e.g., `[https://api.example.com](https://api.example.com)`), sends an HTTP payload, and waits for a response. |
| **Under the Hood** | Built on `hyper`'s server engine and `tower` routing middleware. | Built on `hyper`'s client engine with connection pooling, TLS/SSL, and redirect handling. |
| **Can it replace the other?** | **No.** Axum cannot dial an external URL to deliver data. | **No.** Reqwest cannot bind to a port or listen for incoming client traffic. |

### How Both Are Used in This Project

In the Webhook Relay system, both tools are required at different stages of the pipeline:






**1. Where `axum` is used (`services/ingest-api` & `services/stream-bridge`)**

- `ingest-api` acts as a **listener**. It binds to a port and exposes routes like `POST /api/v1/events`. When a client uses `curl` or an internal service pushes an event, Axum handles the incoming TCP connection, parses the JSON body, and responds with `202 Accepted`.
- `stream-bridge` uses Axum to host the `GET /events/stream` Server-Sent Events (SSE) endpoint that web browsers connect to.

**2. Where `reqwest` is used (`services/dispatch-worker`)**

- The `dispatch-worker` does not accept inbound web traffic, so it doesn't need to listen on any port.
- Instead, it acts as a **caller**. It reads pending destination URLs (like `[https://hooks.slack.com/](https://hooks.slack.com/)...` or `[https://api.merchant.com/webhook](https://api.merchant.com/webhook)`) from PostgreSQL and uses `reqwest::Client` to initiate outgoing HTTP `POST` requests, transmit HMAC signatures in headers, handle TLS handshakes, and record response status codes and network latency.

Would you like to see how to configure connection pooling and request timeouts on the `reqwest::Client` so workers don't leak sockets under high load?

---